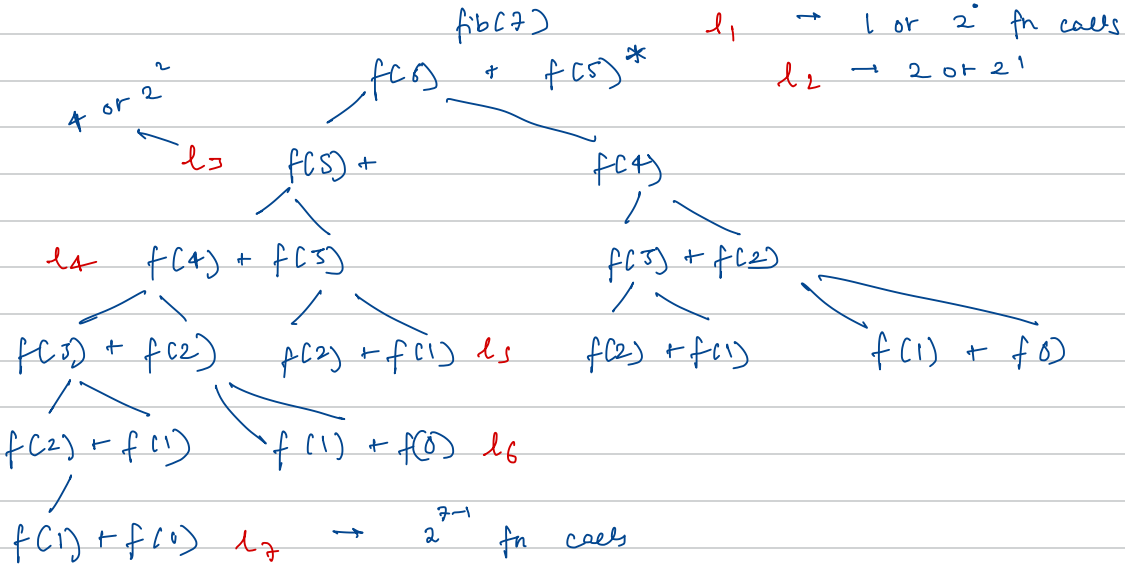
Recursion

→ While calculating factorial, it is recommended to use the long datatype

fibonacci by default, fib(0) = 0 & fib(1) = 1

fib(n) = 0 , 1 , 1 , 2 , 3 , 5 , 8 , 13

0 1 2 3 4 5 6 7



→ n^th fib number has 'n' levels

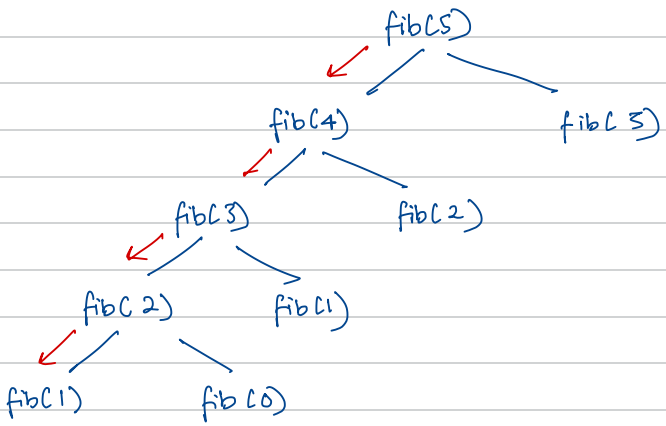
Σ fn calls = $2^0 + 2^1 + 2^2 + .. + 2^{n-1}$ ⇒ Sum of GP of n terms

$= \frac{a(1-r^n)}{1-r} = \frac{2^0 \times (1-2^n)}{(1-2)} = 2^n - 1$

→ Even though the total number of function calls is $2^n - 1$, the space complexity is $O(n)$. Why!

⇒ Space complexity measures memory used at one time, not the total amount of work done over time

At one time for fib(n), there are at max 'n' levels present



→ At any moment, recursⁿ follows one path down the call tree

→ Time complexity however, is $O(2^n)$

NOTE for recursion

S.C. → recursion stack space + other data structures used

↓

only a single path from root to leaf will execute